

EECS 182 HW3 – NotebookLM-Based Final Review Tool

Participation (E): AI-Enhanced Learning for a Single Homework

John Wang

December 7, 2025

1 Overview

This report describes an AI-enhanced learning tool I built for HW3 using Google’s NotebookLM in “notebook” mode. The goal is to support post-homework, pre-final review of HW3 by combining three stages:

1. **Slide generation** from the HW3 PDF and official solutions to create a concise concept map of the homework.
2. **Audio overview** that walks through the slides in 5–10 minutes, providing a multi-modal recap of key ideas.
3. **Interactive problem-centered dialogue** where I ask follow-up questions about specific HW3 problems and the model explains the official solutions step by step.

I share the exact prompts used for slide and audio generation and outline how classmates can reproduce this tool and use it to review HW3. The interaction trace I collected is annotated to highlight both helpful explanations and places where the model’s answers are oversimplified or potentially misleading.

2 Resources and Platform

Platform

All interactions were conducted in **NotebookLM** (notebook mode).

Resources (Uploaded to NotebookLM)

For this HW3 review notebook, I uploaded the following resources:

- **HW3.pdf** – the HW3 problem set.
- **HW3_solutions.pdf** – the official HW3 solutions provided by the course staff.
- **HW3_review_slides.pptx** – a short HW3 review slide deck I created from the official solutions, grouping problems by concept.

Classmates can recreate the same setup by creating a new NotebookLM notebook and uploading their own copies of the HW3 and solution PDFs from the course website, plus the shared review slides.

3 Phase 1: Generating Review Slides from HW3 + Solutions

In the first phase, I used NotebookLM to synthesize a slide-style outline of HW3 based on the homework and official solutions. The goal was to obtain a compact “concept map” for HW3, organized by topic rather than by question number.

Within the notebook, after uploading HW3.pdf and HW3_solutions.pdf (and optionally my initial slide deck), I issued the following prompt to generate a review slide outline.

Slide-Generation Prompt

You are an EECS 182 HW3 review assistant.

Using ONLY the uploaded HW3 PDF, the official HW3 solutions, and the review slides I created, please generate a slide-style summary of HW3.

Requirements:

- Group HW3 into main topics (for example):
 - * Gaussian initialization and RMS norms,
 - * sign-SGD updates and feature changes,
 - * maximal update parameterization (P) and width scaling,
 - * policy gradient and reparameterization.
- For each topic, create 12 slides worth of bullet points that:
 - * name the key concepts being tested,
 - * state the main equations or relationships,
 - * highlight one or two common pitfalls
(e.g., mixing up fan-in vs fan-out,
misusing RMS vs L2 norms,
confusing score function with reparameterization).

The goal is to obtain a compact HW3 concept map that I can skim before the final.

NotebookLM responded with a slide-style outline, grouped by topic. I used this outline to refine my HW3 review slides (HW3_review_slides.pptx), which serve as a visual summary of the core ideas behind each HW3 problem.

4 Phase 2: Generating an Audio Overview from the Slides

In the second phase, I re-uploaded the finalized review slides to the same NotebookLM notebook as an additional resource, and then asked NotebookLM to produce an audio overview that walks through the slides.

The idea is to get a 5–10 minute, low-detail audio recap of HW3 that I can listen to while following along with the slides.

Audio-Generation Prompt

After the review slides were uploaded, I provided the following prompt:

Please generate:

- (1) a slide-style outline for HW3 that summarizes the core concepts behind each problem (12 slides per main topic), and
- (2) an audio overview that walks through these slides in ~510 minutes.

Focus on the high-level ideas:

- Gaussian initialization and RMS norms,
- sign-SGD updates,
- P motivation,
- and the policy gradient / reparameterization questions in HW3.

NotebookLM generated an updated slide-style outline (consistent with my HW3 review slides) and a short audio overview. I selected the low-detail audio option and listened to it once while going through the slides.

This multi-modal review (slides + audio) served as a fast “big-picture recap” of HW3 before I revisited any of the detailed algebra or derivations.

5 Phase 3: Interactive Problem-Centered Dialogue

After completing the slide+audio review, I used the same NotebookLM notebook as an interactive tutor for specific HW3 problems. This phase is where the tool becomes a true AI-enhanced learning environment and where I collected an interaction trace for Participation (E).

Role Prompt for the Dialogue Phase

To focus the model on problem-level explanations, I added the following role prompt in the notebook instructions:

You are now an EECS 182 HW3 problem-level tutor.

I have already:

- reviewed the HW3 concepts using your slide-style summary, and
- listened to the audio overview based on the review slides.

Now I want to go deeper on specific HW3 problems.

For any HW3 question I mention (e.g., "Q1(a)" or "Q3(c)"):

1. First, connect the problem to the high-level concepts from the slides/audio (e.g., Gaussian initialization and RMS norms, sign-SGD updates, P motivation, policy gradient / reparameterization).
2. Then, using ONLY the uploaded HW3 + official solution PDFs:
 - Walk through the official solution step by step.
 - Explain what each step is doing and why it is valid.
 - Explicitly point out common pitfalls or confusions (for example, mixing up fan-in vs fan-out, dropping constants incorrectly,

or confusing score-function vs reparameterization estimators).

3. Finally, propose 12 exam-style variants that test the same idea but with different numbers or wording, and only give hints unless I explicitly ask for full solutions.

If the sources are not explicit enough to answer my question, say so instead of guessing.

Example Interaction: HW3 Q1 Concept Summary

As an example of the dialogue phase, I asked NotebookLM to explain what HW3 Question 1 is really about and which review slides are most relevant. My prompt was:

“Please summarize what HW3 Question 1 is testing conceptually, and tell me which topics my review slides need to cover for this question.”

NotebookLM responded with the following explanation (edited only for minor formatting):

NotebookLM Response (Concept Summary for HW3 Q1). High-Level Focus. HW3 Question 1 is focused on the core concepts of Maximal Update Parameterization (P) and parameter initialization necessary for stable training dynamics in deep neural networks.

Key Concepts Tested by HW3 Q1

1. Normalization and Initialization Scaling (Variance Preservation).

Part (a) tests understanding of how the magnitude (expected squared RMS norm) of output features scales with layer dimensions at initialization, in particular the input dimension d_1 (fan-in).

- It requires calculating the expected squared RMS norm of the output features $y = Wx$ when weights W and inputs x are independently sampled from unit Gaussians.
- The problem shows that to ensure $\mathbb{E}[\text{RMS}(Wx)^2] = 1$ regardless of d_1 and d_2 (output dimension), the weights W must be scaled by $1/\sqrt{d_1}$. This is the Xavier-style initialization scaling that preserves variance across layers.

2. Maximal Update Parameterization (P) Scaling.

Part (b) explores how to maintain a constant rate of feature update during training, regardless of layer width.

- Using a simplified optimizer (SignGD), the problem requires determining how the expected squared RMS norm of the change in features scales with dimension, and shows that it scales as $\Theta(d_1^2)$.
- This leads to the key result that to ensure the expected squared RMS norm of the feature update is 1, the update must be scaled by $1/d_1$. This is the P scaling rule for this setting.

3. Dimensional Dependence of Updates.

Part (c) tests why the update scaling derived in P (here $1/d_1$) depends only on the input dimension d_1 and not on the output dimension d_2 (or the width m).

- The underlying insight is that the parameters affecting a single output feature are independent of other output features; therefore, controlling the RMS norm can be done by considering each output feature independently. The layer width does not affect the required scaling when viewed per-feature.

4. Optimization as Constrained Optimization.

The introduction notes that these scalings relate to viewing SGD and Adam as constrained optimization problems defined by norms over the parameter space. The question encourages connecting the explicit scaling rules to this broader perspective.

Most Relevant Review Deck Topics for Q1

Since NotebookLM did not see the exact slide numbers in my deck, it instead listed the topics the slides should cover for Q1:

- **Maximal Update Parameterization (P).**

Principles and derivations showing how P preserves feature learning as network width changes, linked to the Tensor Programs V paper on zero-shot hyperparameter transfer.

- **Initialization Schemes.**

The need for variance stabilization and the derivation of Xavier/He initialization, ensuring that output variance remains constant across layers, typically via scaling by $1/\text{fan-in}$.

- **Scaling Behavior in Wide Networks.**

How network width affects learning dynamics and how different parameterizations (standard vs. P) change training-loss vs learning-rate curves (e.g., as in Figure 1 of the P paper).

- **Relationship Between Optimization and Parameter Norms.**

How optimizers like Adam use dimension-dependent scaling factors (such as $1/\text{fan-in}$) and how this connects to constrained optimization viewpoints discussed in lecture.

- **Feature Scaling Desiderata.**

Desirable properties for feature learning, such as ensuring each hidden unit moves by $O(1)$ and stays $O(1)$ during training. This relates to “Desideratum 1” and spectral conditions for feature learning discussed in the reading.

My Annotation on This Interaction

In my annotated trace, I highlighted the following points:

- **Helpful aspects.**

- The response cleanly organizes HW3 Q1 into four conceptual components: initialization scaling, P scaling, dimensional dependence, and optimization-as-constrained-optimization.
- It correctly captures the main scaling results ($\mathbb{E}[\text{RMS}(Wx)^2]$ depending on d_1 and the $\Theta(d_1^2)$ growth of feature updates under SignGD), and why the resulting P scaling is $1/d_1$ in this setup.
- The list of review topics is a good checklist for what my HW3 slides should emphasize before the final.

- **Potentially misleading aspects.**

- Some of the wording could be read as suggesting that the $1/d_1$ P scaling is universally correct, whereas in reality it depends on the architecture and activation assumptions used in HW3.
- The description of “optimization as constrained optimization” is high-level; without lecture context a first-time reader might not fully understand the norm interpretation.

I repeated similar interactions for other HW3 problems (e.g., the policy gradient / reparameterization question), following the same pattern: concept summary, guided walkthrough of the official solution, and exam-style variants, each with my own annotations.

Sources

+ Add sources

Try Deep Research for an in-depth report and new sources!

Search the web for new sources

Web

Fast Research

→

Select all sources ☐

Deep_Learning_Engineer_Toolkits_Final_Review.pdf ☒

EECS 182 Conceptual Mastery and Review Protocol ☐

hw03_solution.pdf ☒

Chat

concept that Stochastic Gradient Descent (SGD) and Adam can be viewed as constrained optimization problems defined by a norm over the parameter space ¹.

Most Relevant Review Deck Concepts/Slides

Since the specific slides in your review deck are unavailable, the most relevant topics they would need to cover, based on the material in Question 1, are:

- **Maximal Update Parameterization (μ P):** Specifically, the principles and derivation showing how μ P ensures feature learning preservation as network width changes ¹, ⁷, ². This might be linked to discussions of the paper *Tensor Programs V: Tuning Large Neural Networks via Zero-Shot Hyperparameter Transfer* ⁷.
- **Initialization Schemes:** The slides covering the need for variance stabilization and the derivation of **Xavier initialization** or similar initialization schemes that ensure output variance remains constant across layers, typically scaling by $1/\text{fanin}$ ⁴, ⁵, ⁸.
- **Scaling Behavior in Wide Networks:** Material that discusses the implications of network width on learning dynamics and how different parameterizations (standard vs. μ P) affect the training loss vs. learning rate curves, as seen in Figure 1 of the μ P paper ⁸.
- **Relationship between Optimization and Parameter Norms:** Slides that discuss how optimization algorithms like Adam use scaling factors based on dimension (like $1/\text{fanin}$) and how these relate to constrained optimization ¹, ⁸. The homework suggests that the elementary argument supporting the $1/\text{fanin}$ scaling for Adam LR for hidden layers was given in lecture ⁸.
- **Feature Scaling Desiderata:** Slides covering desirable properties for feature learning, such as **Desideratum 1** (ensuring each element of the hidden vector moves by $O(1)$ and maintains a size of $O(1)$ during training) ⁹. This concept is linked to the Spectral Condition for Feature Learning paper ¹⁰.

Save to note

Studio

Audio Overview

Video Overview

Mind Map

Reports

Flashcards

Quiz

Infographic

Slide Deck

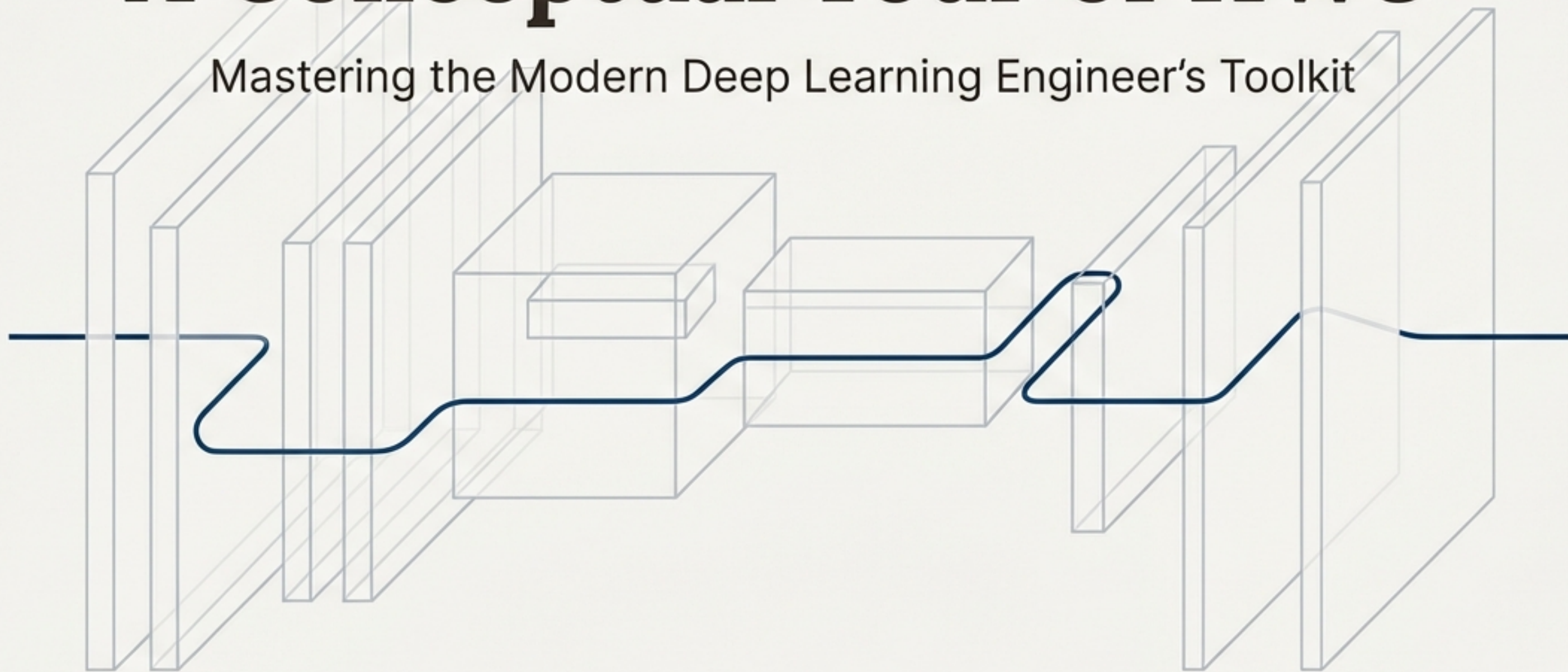
Scaling Deep Learning Statistically and...
Deep Dive · 2 sources · 1m ago

Deep Learning Engineer Toolkits Final Review
1 source · 20m ago

EECS 182 Conceptual Mastery and Review Protocol
22m ago

EECS 182 Final Review: A Conceptual Tour of HW3

Mastering the Modern Deep Learning Engineer's Toolkit



Your Three Essential Toolkits for Building and Training Large Networks



1. Principled Scaling

The art of making models width-agnostic with Xavier & μ P.

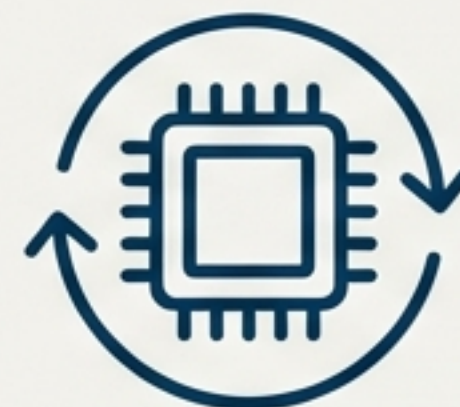
***Case Studies: HW3 Q1, Q2, Q3**



2. Advanced Gradient Estimation

How to train models on probability distributions.

***Case Study: HW3 Q4**



3. Systems at Scale

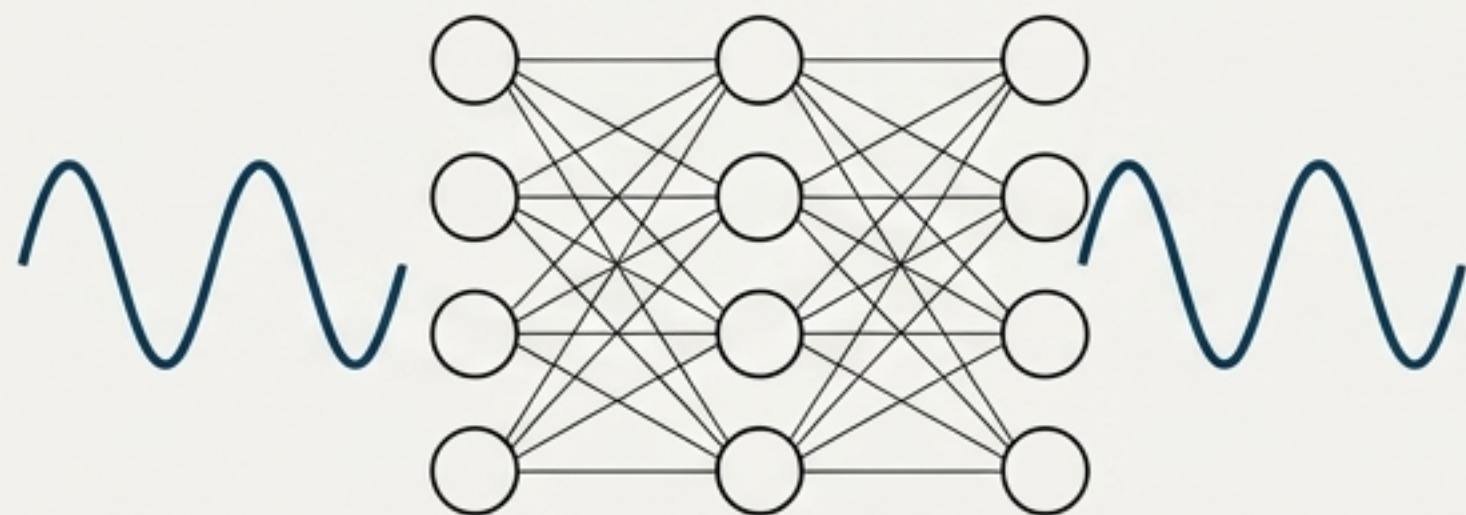
Trading compute for memory when your model is too big.

***Case Study: HW3 Q5**

Toolkit #1: The Billion-Dollar Question of Scaling

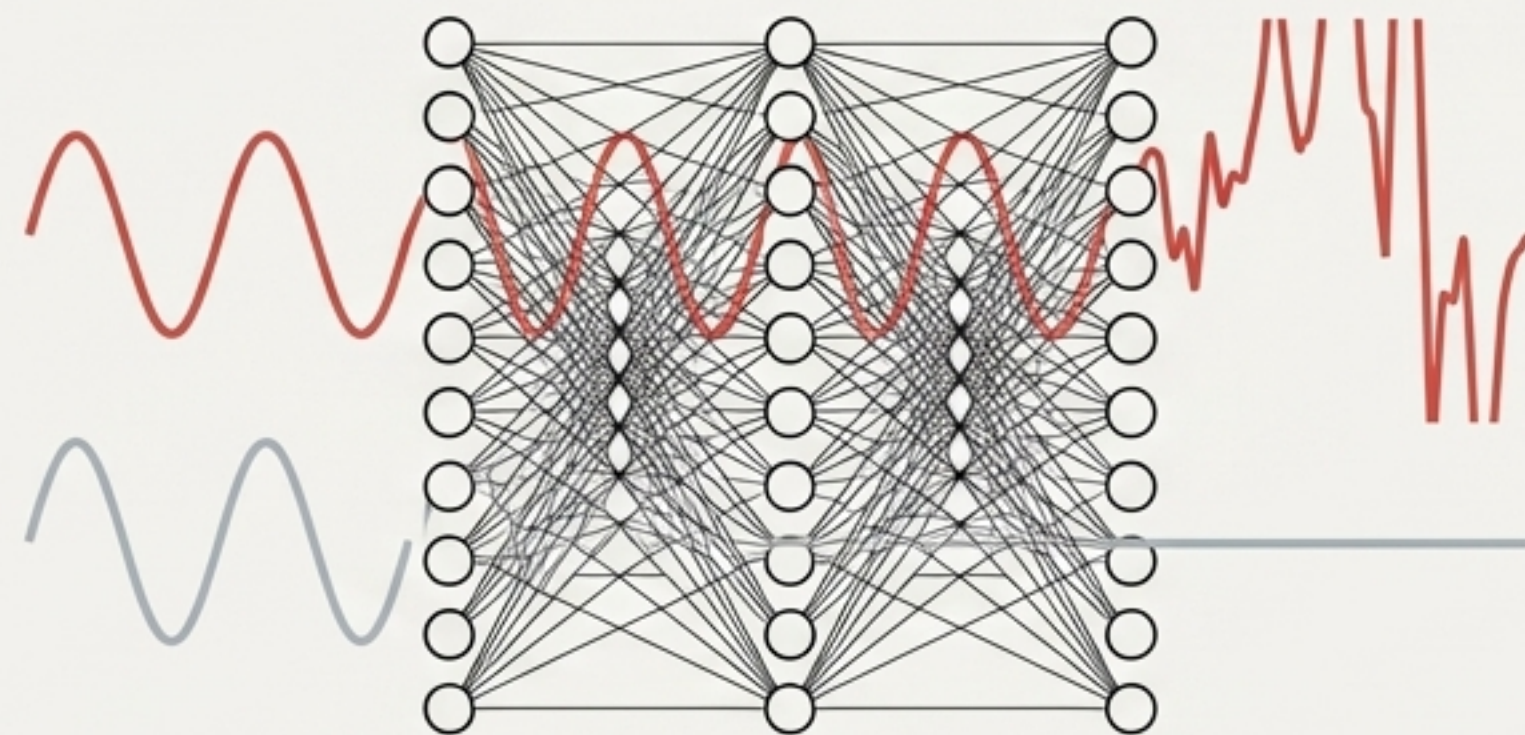
Why do hyperparameters that work for a small model completely fail on a large one?

Narrow Network



Stable Training

Wide Network



Exploding / Vanishing Activations

Without principled scaling, the behavior of a network changes dramatically with its width, making hyperparameter tuning an expensive, unrepeatable art.

The Core Insight: Control the Signal, Not Just the Weights

To make training stable and width-agnostic, we must ensure the “signal” (the features) maintains a constant variance throughout the network's life. This requires a two-pronged approach:

1. At Initialization (HW3 Q1a): We use **Xavier Initialization** to scale weights so that the output features of a layer have a constant variance (e.g., 1) from the very first forward pass.

2. During Training (HW3 Q1b, Q3): We use **Maximal Update Parameterization (μ P)** to scale our optimizer's updates (e.g., the learning rate) so that the *change* in output features has a constant magnitude. This prevents wider layers from receiving disproportionately large or small updates.

t=0: Initialization

 **Xavier Scaling**

Ensures initial output variance is constant.

t>0: Training

 **μ P Scaling**

Ensures feature *update* magnitude is constant.

Case Study: The Logic of Xavier Scaling (HW3 Q1a)

1. **Goal:** For an output feature $y_i = \sum W_{ij}x_j$, we want the expected squared norm to be constant, i.e., $\mathbb{E}[y_i^2] = 1$.
2. **The Math:** Since inputs and weights are zero-mean, $\mathbb{E}[y_i^2] = \text{Var}(y_i) = \text{Var}(\sum W_{ij}x_j)$.
3. **Key Assumption:** Weights W and inputs x are **independent**. This allows us to state that variance is both additive and multiplicative: $\text{Var}(\sum W_{ij}x_j) = \sum \text{Var}(W_{ij}x_j) = \sum \text{Var}(W_{ij})\text{Var}(x_j)$
4. **The Result:** If $\text{Var}(W_{ij})=1$ and $\text{Var}(x_j)=1$, then $\text{Var}(y_i) = d_{in} * 1 * 1 = d_{in}$. The output variance scales linearly with the input dimension (fan_in).
5. **The Fix:** To make $\text{Var}(y_i) = 1$, we must scale W by a factor c such that $\text{Var}(cW) * d_{in} = 1$. Since $\text{Var}(cW) = c^2\text{Var}(W) = c^2$, we need $c^2 * d_{in} = 1$, which means $\mathbf{c} = \frac{1}{\sqrt{d_{in}}}$.

Check for Understanding

Based on this derivation, what would happen to our scaling factor if the input features `x` were known to have a variance of 4 instead of 1?

Case Study: The Logic of μ P Update Scaling (HW3 Q1b)

1. **Goal:** Ensure the magnitude of the *change* in features, $y' = \text{sign}(gix^T)x_i$, is constant regardless of width.
2. **The Update:** For a single output feature, the update is $y' = g * \text{sign}(x)^T x = g * \sum |x_j|$.
3. **Crucial Pitfall:** Unlike initialization, the update $\text{sign}(gix^T)$ is now **correlated** with the input x ! We cannot simply multiply variances. The term $\text{sign}(x)^T x$ simplifies to $\sum |x_j|$.
4. **The Result:** Each term $|x_j|$ has a constant positive expected value. Since there are d_{in} such terms, $\mathbb{E}[y']$ scales with d_{in} . The expected *squared* norm therefore scales as $\Theta(d_{in}^2)$.
5. **The Fix:** To make the expected squared norm of the feature update constant (e.g., 1), we must scale the entire update by $1/d_{in}$. This is the core of μ P for SignGD/Adam-style optimizers.

The shift from **independence** (at init) to **correlation** (during training) is the reason for the different scaling factors ($1/\sqrt{d_{in}}$ vs $1/d_{in}$).

Mastery Challenge: Apply Your Knowledge of Scaling

New Activation Function

You are designing a new activation function, “Double ReLU,” defined as $f(z) = 2 * \max(0, z)$. If your inputs $\mathbf{x}$ and initial weights $\mathbf{W}$ are standard Gaussians, how must you adjust the standard Xavier initialization scaling of $1/\sqrt{d_{in}}$ to maintain unit variance of the output features after this activation?

Hint: Think about how $f(z)$ affects the variance of the output.

New Optimizer Rule

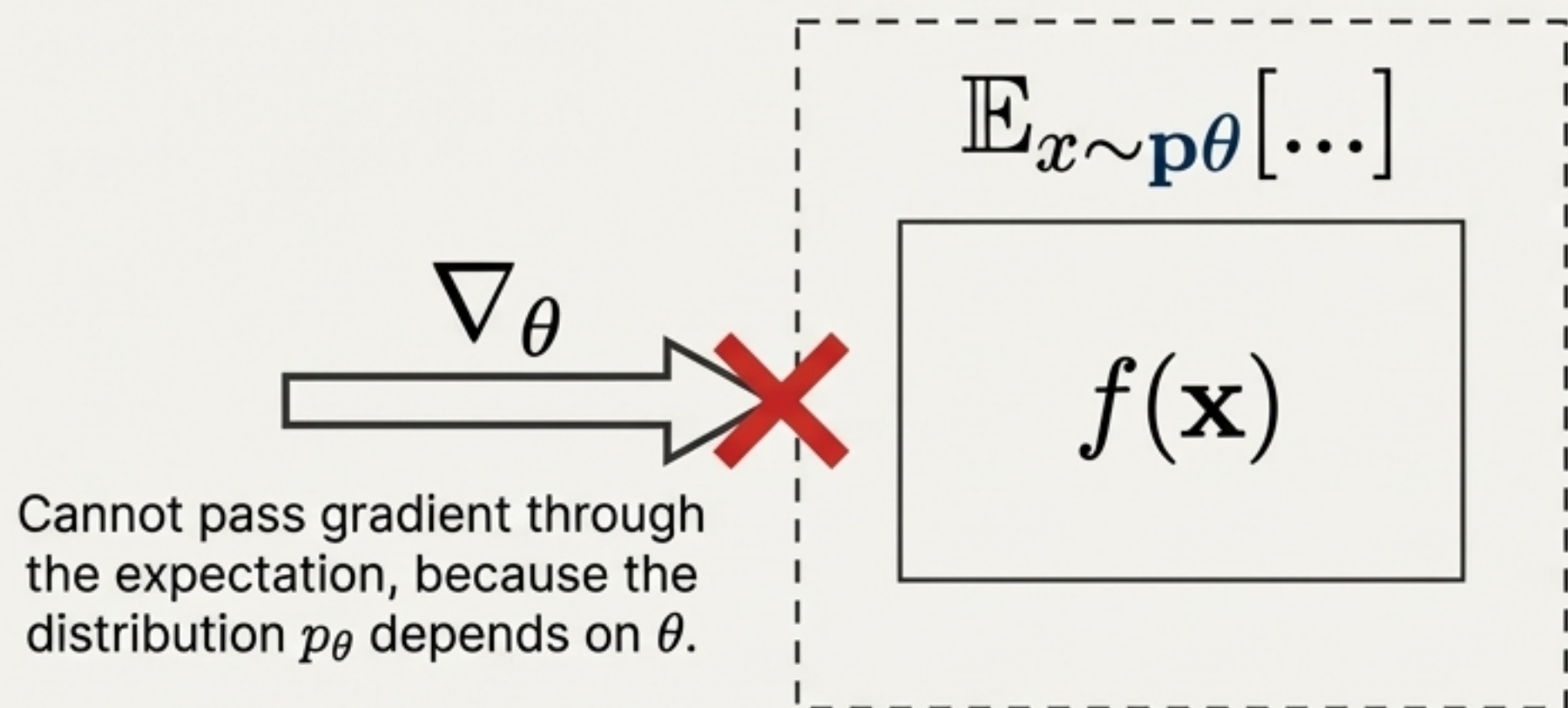
An engineer proposes a new optimizer update rule for a single weight W_i that scales with $(x_i)^2$ instead of $\text{sign}(x_i)$. To maintain the principles of μP , how should the learning rate for this new optimizer scale with the input dimension d_{in} ?

Hint: Calculate the expected magnitude of the feature update y' under this new rule.

Toolkit #2: How to Optimize the Unknown

How do you compute a gradient when your parameters define the very probability distribution you're sampling from?

$$F(\theta) = \mathbb{E}_{x \sim \mathbf{p}\theta}[f(\mathbf{x})]$$



This problem appears everywhere from reinforcement learning to training variational autoencoders. We need a way to get θ out from “inside” the expectation.

Two Methods for Gradient Estimation (HW3 Q4)

HW3 Q4 introduces two powerful methods for finding $\nabla_{\theta} \mathbb{E}_{x \sim p_{\theta}}[f(x)]$.

1. The Score Function Method (aka REINFORCE)

Concept (from Q4d): Uses the 'log-derivative trick' ($\nabla_p = p \nabla \log p$) to rewrite the gradient.

Resulting Estimator:

$$\mathbb{E}_{x \sim p_{\theta}}[f(x) * \nabla_{\theta} \log p_{\theta}(x)]$$

Pros/Cons: Broadly applicable (doesn't require f to be differentiable), but can have **high variance**.

2. The Reparameterization Trick

Concept (from Q4b,c,e): Rewrites the random variable x as a deterministic, differentiable function of θ and a base noise variable z , i.e., $x = g(z, \theta)$.

Resulting Estimator: $\mathbb{E}_z[\nabla_{\theta} f(g(z, \theta))]$

Pros/Cons: Lower variance, but **requires f to be differentiable** and x to be reparameterizable.

Deep Dive: The Reparameterization Trick (HW3 Q4c)

Walkthrough: The Gaussian Case

1. **Original Objective:** $F(\mu) = \mathbb{E}_{x \sim N(\mu, \sigma^2 I)}[f(\mathbf{x})]$
2. **Reparameterize $\mathbf{x}$:** We can rewrite $\mathbf{x}$ as a function of a standard normal variable $\mathbf{z} \sim N(0, I)$. The function is $\mathbf{x} = \mathbf{g}(\mathbf{z}, \mu) = \sigma \mathbf{z} + \mu$.
3. **Rewrite the Expectation:** The objective becomes $F(\mu) = \mathbb{E}_{\mathbf{z} \sim N(0, I)}[f(\sigma \mathbf{z} + \mu)]$. Notice the expectation is now over $\mathbf{z}$, which does *not* depend on μ .
4. **Take the Gradient:** We can now move the gradient $\nabla \mu$ inside the expectation:
 $\nabla \mu F(\mu) = \mathbb{E}_{\mathbf{z} \sim N(0, I)}[\nabla_{\mu} f(\sigma \mathbf{z} + \mu)]$.
5. **Apply Chain Rule:** The final estimator is $\mathbb{E}_{\mathbf{z} \sim N(0, I)}[\nabla_{\mathbf{x}} f|_{\mathbf{x}=\sigma \mathbf{z} + \mu}]$.

The Key Idea in Plain Language

We are separating the parameters (μ) from the randomness ($\mathbf{z}$). The 'dice roll' ($\mathbf{z}$) is now fixed and independent of our parameters, allowing the gradient to flow through the deterministic function g as if it were a normal part of the computation graph.

Mastery Challenge: Reparameterize a New Distribution

Problem

Let x be drawn from an **Exponential distribution** with rate parameter λ . Its probability density function is $p(x; \lambda) = \lambda e^{-\lambda x}$. This distribution can be reparameterized by first sampling $u \sim \text{Uniform}(0,1)$ and then transforming it via the inverse CDF: $x = -\ln(u)/\lambda$.

Task: Using this reparameterization, derive the gradient estimator for $\nabla_{\lambda} \mathbb{E}[f(x)]$.

Check for Understanding

After you derive the estimator, why might this be a lower-variance way to estimate the gradient compared to using the score function method ($\mathbb{E}[f(x) * \nabla_{\lambda} \log p(x; \lambda)]$) for this distribution?

Toolkit #3: Your Model is Brilliant. But It Won't Fit on the GPU.

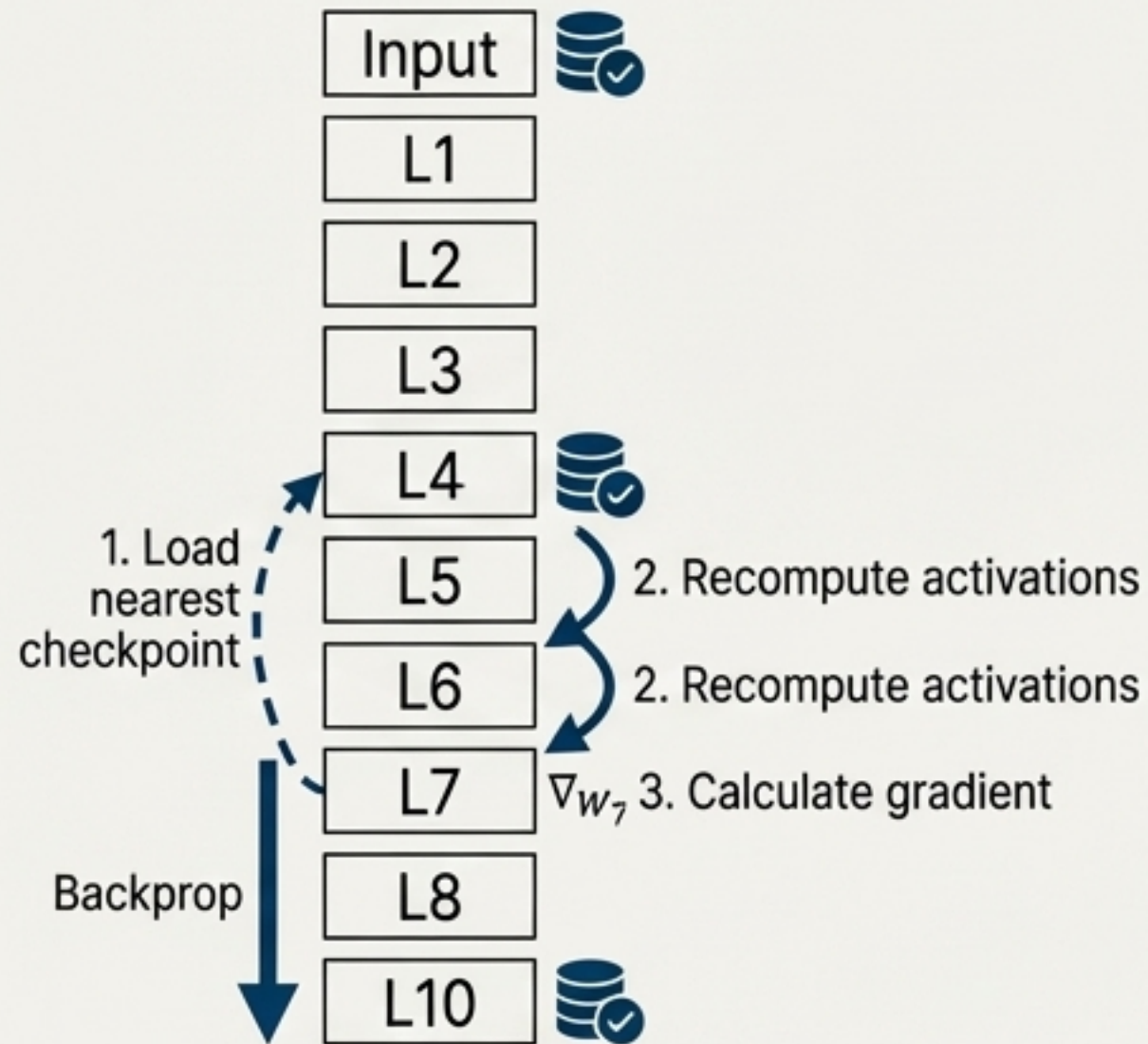
The Memory Wall



In large models, the memory required to store every layer's activation for the backward pass often exceeds the capacity of the hardware. This is a fundamental bottleneck to scaling.

Solution: Tensor Rematerialization (Gradient Checkpointing) (HW3 Q5)

Tensor Rematerialization is a technique that intentionally discards intermediate activations during the forward pass to save memory. During the backward pass, it recomputes them on-the-fly from the nearest stored checkpoint. It is a direct trade: we accept higher computational cost (more `fwd` ops) in exchange for a dramatically reduced memory footprint.



Scenario: 10-layer network, checkpoints at Input, L5, and L10.

Task: Compute gradient for Layer 7.

1. The backpropagation process reaches L7.
2. Load the activations from the *nearest previous checkpoint*: Layer 5.
3. Recompute forward: Run a fwd pass on Layer 6, then Layer 7, to get the needed activations.
4. Use the recomputed activations to calculate ∇W_7 . Discard recomputed activations.

Common Pitfall

A common mistake is assuming you always recompute from the very first layer. To compute the gradient for Layer 4, it is more efficient to recompute from the initial input (4 fwd ops) than to load from Layer 5 and try to work backward.

The Tradeoff in Practice: A Quantitative Look (HW3 Q5c)

How fast would loading from disk have to be to match the performance of tensor rematerialization?

Knowns (from HW3)

- fwd operation time: 20 ns
- loadmem operation time: 10 ns

Tensor Rematerialization

Total backward pass requires 20 fwd operations and 10 loadmem operations.

$$\begin{aligned}\text{Total Time} &= (20 * 20 \text{ ns}) + (10 * 10 \text{ ns}) \\ &= 400 \text{ ns} + 100 \text{ ns} \\ &= \mathbf{500 \text{ ns}}\end{aligned}$$

Disk-Based Method

Requires loading 10 activations from disk.

$$\text{Total Time} = \mathbf{10} * t_{\text{loaddisk}}$$

The Breakeven Point

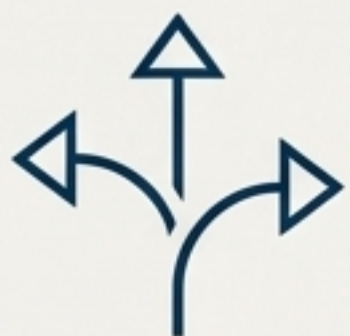
$$\begin{aligned}10 * t_{\text{loaddisk}} &= 500 \text{ ns} \\ t_{\text{loaddisk}} &= \mathbf{50 \text{ ns}}\end{aligned}$$

To be competitive, a disk load operation would need to take **50 ns**. Given that typical SSD access times are in the range of 10,000-100,000 ns, tensor rematerialization is vastly more practical than storing all activations to disk for large models.

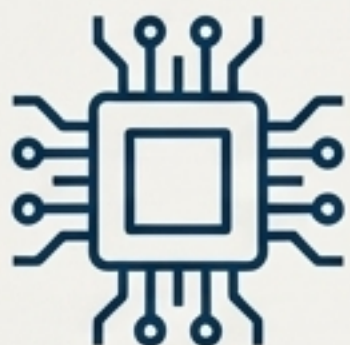
Your Upgraded Toolkit for the Final Exam



1. Principled Scaling is Mandatory. To build networks that are robust to width changes, you must control variance. Use Xavier ($1/\sqrt{d_{in}}$ scaling) at initialization and μP ($1/\sqrt{d_{in}}$ LR scaling) during training to manage the variance of features and their updates.



2. Gradients Through Expectations are Solvable. You have two tools to optimize objectives of the form $\mathbb{E}[f(x)]$. Use the Score Function (log-derivative trick) when your function is a black box, and the lower-variance Reparameterization Trick when you can express your random variable as a deterministic, differentiable function of your parameters.



3. Memory is a Tradeoff, Not a Hard Limit. Use Tensor Rematerialization (gradient checkpointing) to trade computational cycles for memory. This allows you to train models far larger than what would normally fit on your GPU.

“These toolkits are not just answers to homework problems; they are fundamental principles for engineering modern, large-scale neural networks.”